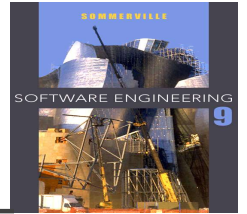


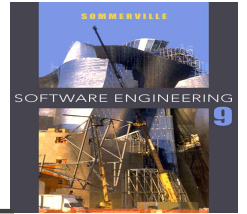
Software Testing- Part II

Release testing



- ✧ **Release testing** is the process of testing a particular release of a system that is intended for use outside of the development team.
- ✧ **The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use.**
 - **Release testing**, therefore, has to show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use.
- ✧ **Release testing is usually a black-box testing process where tests are only derived from the system specification.**

Release testing and system testing

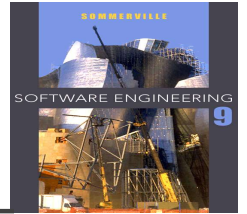


✧ Release testing is a form of system testing.

✧ Important differences:

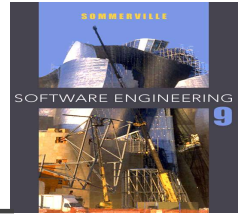
- A separate team that has not been involved in the system development, should be responsible for release testing.
- System testing by the development team should focus on discovering bugs in the system (**defect testing**). The objective of release testing is to check that the system meets its requirements and is good enough for external use (**validation testing**).

Requirements based testing



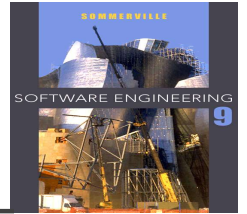
- ✧ Requirements-based testing involves examining each requirement and developing a test or tests for it.
- ✧ MHC-PMS requirements:
 - If a patient is known to be allergic to any particular medication, then prescription of that medication shall result in a warning message being issued to the system user.
 - If a prescriber chooses to ignore an allergy warning, they shall provide a reason why this has been ignored.

Requirements tests



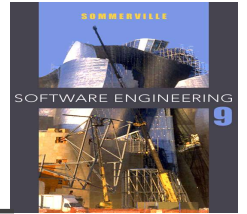
- ✧ Set up a patient record with no known allergies. Prescribe medication for allergies that are known to exist. Check that a warning message is not issued by the system.
- ✧ Set up a patient record with a known allergy. Prescribe the medication to that the patient is allergic to, and check that the warning is issued by the system.
- ✧ Set up a patient record in which allergies to two or more drugs are recorded. Prescribe both of these drugs separately and check that the correct warning for each drug is issued.
- ✧ Prescribe two drugs that the patient is allergic to. Check that two warnings are correctly issued.
- ✧ Prescribe a drug that issues a warning and overrule that warning. Check that the system requires the user to provide information explaining why the warning was overruled.

Performance testing



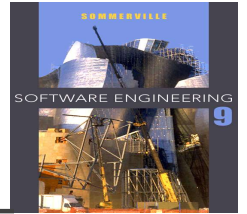
- ✧ Part of release testing may involve testing the emergent properties of a system, such as **performance** and **reliability**.
- ✧ Tests should reflect the profile of use of the system.
- ✧ **Performance tests** usually involve planning a series of tests where the load is steadily increased until the system performance becomes unacceptable.
- ✧ **Stress testing** is a form of performance testing where the system is deliberately overloaded to test its failure behaviour.

User testing



- ✧ User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- ✧ **User testing is essential**, even when comprehensive system and release testing have been carried out.
 - The reason for this is that influences from the user's working environment have a major effect on the reliability, performance, usability and robustness of a system. These cannot be replicated in a testing environment.

Types of user testing



✧ Alpha testing

- Users of the software work with the development team to test the software at the developer's site.

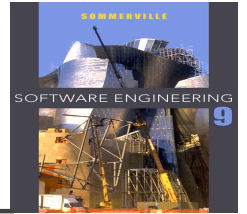
✧ Beta testing

- A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.

✧ Acceptance testing

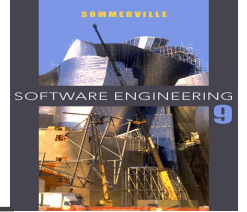
- Customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment. Primarily for custom systems.

Agile methods and acceptance testing



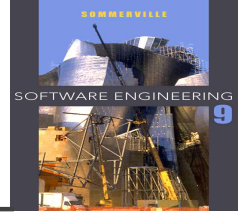
- ✧ In agile methods, the user/customer is part of the development team and is responsible for making decisions on the acceptability of the system.
- ✧ Tests are defined by the user/customer and are integrated with other tests in that they are run automatically when changes are made.
- ✧ There is no separate acceptance testing process.
- ✧ Main problem here is whether or not the embedded user is 'typical' and can represent the interests of all system stakeholders.

Static and dynamic verification



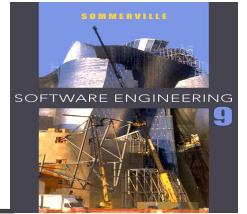
- ✧ **Software inspections.** Concerned with analysis of the static system representation to discover problems (static verification)
 - May be supplemented by tool-based document and code analysis
- ✧ **Software testing.** Concerned with exercising and observing product behaviour (dynamic verification)
 - The system is executed with test data and its operational behaviour is observed

Software inspections



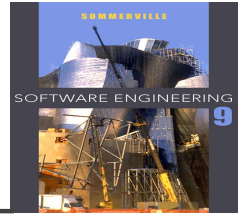
- ✧ These involve people examining the source representation with the aim of discovering anomalies and defects.
- ✧ Inspections do not require execution of a system, so may be used before implementation.
- ✧ They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
- ✧ They have been shown to be an effective technique for discovering program errors.

Inspection success



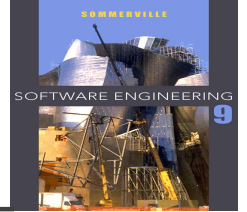
- ✧ Many different defects may be discovered in a single inspection. In testing, one defect may mask another so several executions are required.
- ✧ They reuse domain and programming knowledge so reviewers are likely to have seen the types of error that commonly arise.

Inspections and testing



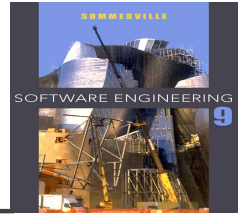
- ✧ Inspections and testing are complementary and not opposing verification techniques.
- ✧ Both should be used during the V & V process.
- ✧ Inspections can check conformance with a specification but not conformance with the customer's real requirements.
- ✧ Inspections cannot check non-functional characteristics such as performance, usability, etc.

Program inspections



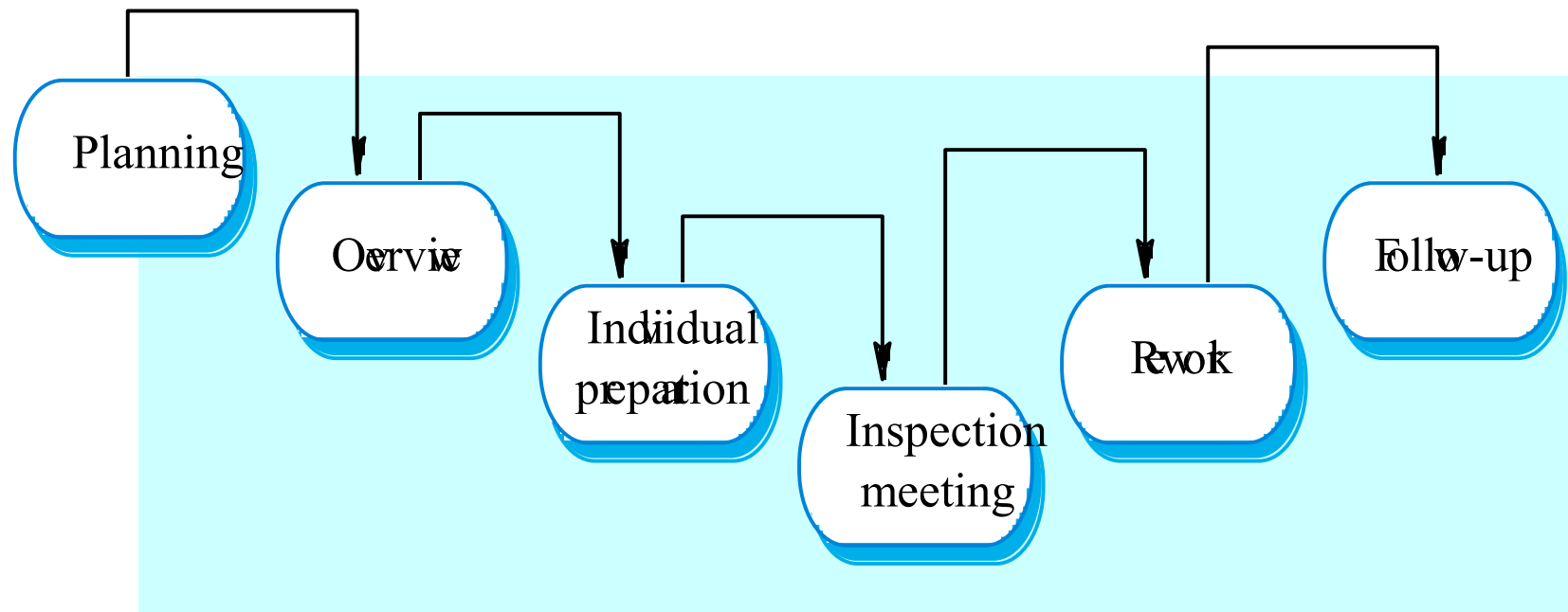
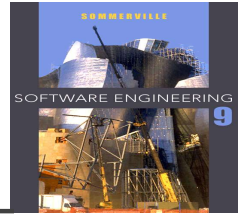
- ✧ Formalised approach to document reviews
- ✧ Intended explicitly for defect detection (not correction).
- ✧ Defects may be logical errors, anomalies in the code that might indicate an erroneous condition (e.g. an uninitialised variable) or non-compliance with standards.

Inspection pre-conditions

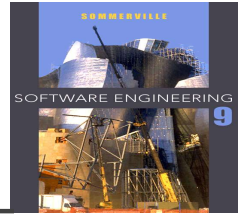


- ✧ A precise specification must be available.
- ✧ Team members must be familiar with the organisation standards.
- ✧ Syntactically correct code or other system representations must be available.
- ✧ An error checklist should be prepared.
- ✧ Management must accept that inspection will increase costs early in the software process.
- ✧ Management should not use inspections for staff appraisal, for ex; finding out who makes mistakes.

The inspection process

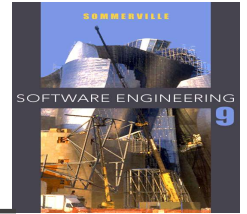


Inspection procedure



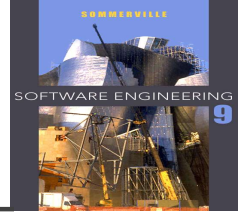
- ✧ System overview presented to inspection team.
- ✧ Code and associated documents are distributed to inspection team in advance.
- ✧ Inspection takes place and discovered errors are noted.
- ✧ Modifications are made to repair discovered errors.
- ✧ Re-inspection may or may not be required.

Inspection roles



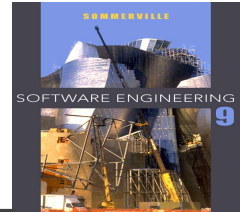
Author or owner	The programmer or designer responsible for producing the program or document. Responsible for fixing defects discovered during the inspection process.
Inspector	Finds errors, omissions and inconsistencies in programs and documents. May also identify broader issues that are outside the scope of the inspection team.
Reader	Presents the code or document at an inspection meeting.
Scribe	Records the results of the inspection meeting.
Chairman or moderator	Manages the process and facilitates the inspection. Reports process results to the Chief moderator.
Chief moderator	Responsible for inspection process improvements, checklist updating, standards development etc.

Inspection checklists



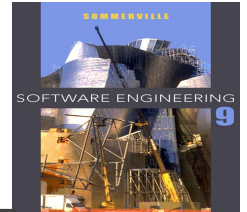
- ✧ Checklist of common errors should be used to drive the inspection.
- ✧ Error checklists are programming language dependent and reflect the characteristic errors that are likely to arise in the language.
- ✧ In general, the 'weaker' the type checking, the larger the checklist.
 - Examples: Initialisation, Constant naming, loop termination, array bounds, etc.

Inspection checks 1



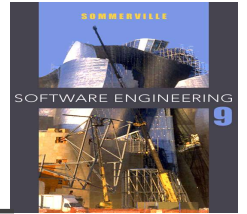
Data faults	Are all program variables initialised before their values are used? Have all constants been named? Should the upper bound of arrays be equal to the size of the array or Size -1? If character strings are used, is a delimiter explicitly assigned? Is there any possibility of buffer overflow?
Control faults	For each conditional statement, is the condition correct? Is each loop certain to terminate? Are compound statements correctly bracketed? In case statements, are all possible cases accounted for? If a break is required after each case in case statements, has it been included?
Input/output faults	Are all input variables used? Are all output variables assigned a value before they are output? Can unexpected inputs cause corruption?

Inspection checks 2



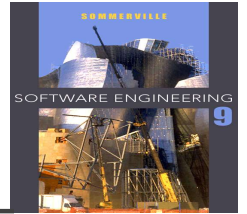
Interface faults	Do all function and method calls have the correct number of parameters? Do formal and actual parameter types match? Are the parameters in the right order? If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	If a linked structure is modified, have all links been correctly reassigned? If dynamic storage is used, has space been allocated correctly? Is space explicitly de-allocated after it is no longer required?
Exception management faults	Have all possible error conditions been taken into account?

Inspection rate



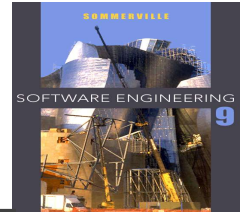
- ✧ 500 statements/hour during overview.
- ✧ 125 source statement/hour during individual preparation.
- ✧ 90-125 statements/hour can be inspected.
- ✧ Inspection is therefore an expensive process.
- ✧ Inspecting 500 lines costs about 40 man/hours effort.

Automated static analysis



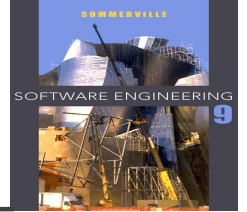
- ✧ Static analysers are software tools for source text processing.
- ✧ They parse the program text and try to discover potentially erroneous conditions and bring these to the attention of the V & V team.
- ✧ They are very effective as an aid to inspections - they are a supplement to but not a replacement for inspections.

Static analysis checks



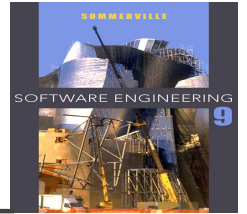
Fault class	Static analysis check
Data faults	Variables used before initialisation Variables declared but never used Variables assigned twice but never used between assignments Possible array bound violations Undeclared variables
Control faults	Unreachable code Unconditional branches into loops
Input/output faults	Variables output twice with no intervening assignment
Interface faults	Parameter type mismatches Parameter number mismatches Non-usage of the results of functions Uncalled functions and procedures
Storage management faults	Unassigned pointers Pointer arithmetic

Stages of static analysis



- ✧ **Control flow analysis.** Checks for loops with multiple exit or entry points, finds unreachable code, etc.
- ✧ **Data use analysis.** Detects uninitialised variables, variables written twice without an intervening assignment, variables which are declared but never used, etc.
- ✧ **Interface analysis.** Checks the consistency of routine and procedure declarations and their use.

Stages of static analysis



- ✧ **Information flow analysis.** Identifies the dependencies of output variables. Does not detect anomalies itself but highlights information for code inspection or review
- ✧ **Path analysis.** Identifies paths through the program and sets out the statements executed in that path. Again, potentially useful in the review process.
- ✧ Both these stages generate vast amounts of information. They must be used with care.